

Solo E-Commerce Platform -- Low Level Design (LLD)

Solo E-Commerce Platform

Document Version 1.0 | March 2026

Document Version: 1.1

Date: 09 May 2026

Author: Solo Engineering Team

Status: Final

1. Purpose

This document drills into module-level design for both backend (NestJS) and frontend (React) tiers: responsibilities, controllers/services/DTOs, key classes, state stores, integration points and notable rules. Pair it with the HLD for system context, the Architecture Document for deployment, and 06 API Reference for the full endpoint catalogue.

> Backend layout: backend/src/<module>/{controller,service,module}.ts

> + dto/. Modules registered in

> backend/src/app.module.ts.

>

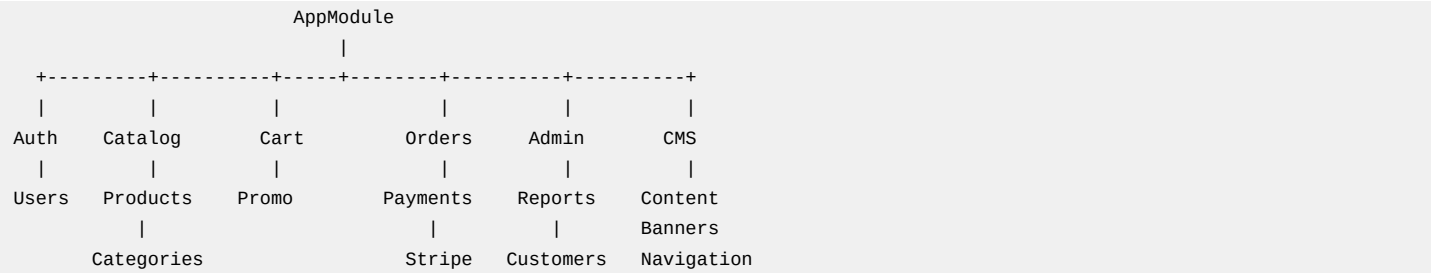
> Frontend layout: `frontend-react/src/{api,stores,hooks,pages, components,lib}` (Vite + React + TypeScript).

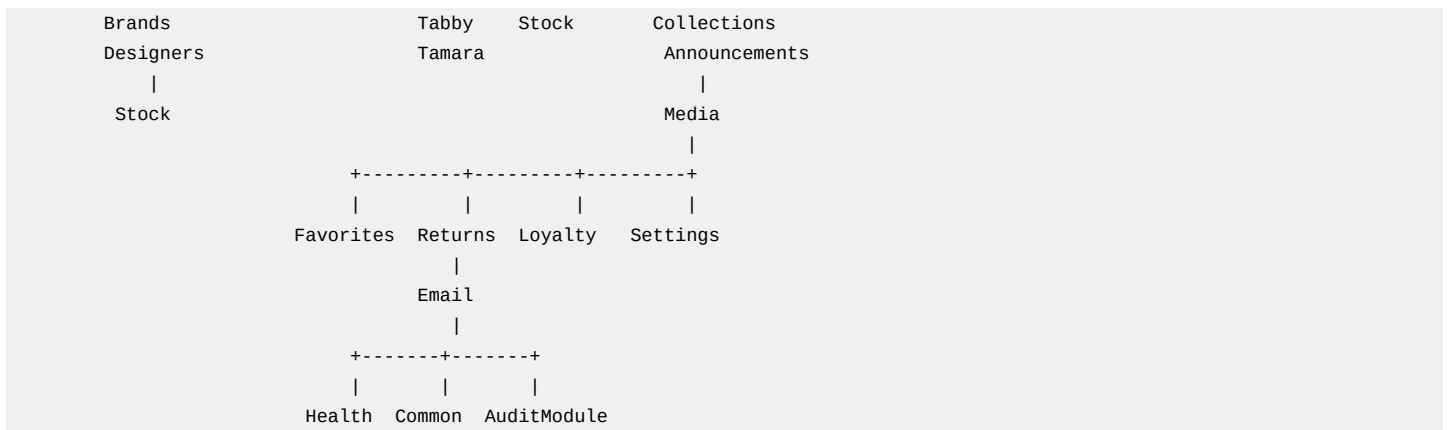
2. Backend Module Design

2.1 Application Bootstrap (main.ts)

- Boot order:
1. tracing.ts initialises Application Insights before any Nest code.
 2. NestFactory.create(AppModule, { bufferLogs: true }).
 3. Pino logger attached.
 4. app.setGlobalPrefix('api') -- every route is /api/....
 5. app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true, forbidNonWhitelisted: true })).
 6. app.useGlobalFilters(new HttpExceptionFilter()).
 7. Helmet, CORS (allowlist from CORS_ORIGINS), compression.
 8. Swagger UI at /api/docs (production-gated by SWAGGER_ENABLED).
 9. Global AuditInterceptor registered via APP_INTERCEPTOR provider.
 10. app.listen(PORT).

2.2 Module Dependency Graph





AuditModule is wired globally (@Global); its AuditInterceptor is registered as APP_INTERCEPTOR so every controller method on every module flows through it (see §7.3).

3. Authentication Module -- Detailed Design

3.1 Class Diagram



3.2 Token Data Structures

Token	Format	TTL	Storage
Access JWT	HS256, payload { sub, role, email, iat, exp }	15 min	client-side (localStorage)
Refresh	Opaque random 64-byte URL-safe string	7 days (configurable)	hashed in refresh_tokens table
Reset / Verify	Single-use signed token	1 hour	password_reset_tokens / email_verification_tokens

refresh_tokens columns: id, userId, tokenHash, expiresAt,

revokedAt, replacedByTokenId, createdAt. Reuse detection: if a revoked token is presented, the entire chain (replacedByTokenId lineage) is revoked and the user is forced back to login.

3.3 Password Hashing

```
// backend/src/auth/password.service.ts (conceptual)
import * as argon2 from 'argon2';

async hash(plain: string): Promise<string> {
  return argon2.hash(plain, {
    type: argon2.argon2id,
    memoryCost: 19_456,    // ~19 MB
    timeCost: 2,
    parallelism: 1,
  });
}

async verify(plain: string, hash: string): Promise<boolean> {
  return argon2.verify(hash, plain);
}
```

Login emits a generic "Invalid credentials" message regardless of whether the email exists or the password is wrong (no user-enumeration leak).

3.4 Rate Limiting (Auth Endpoints)

/api/auth/* is hard-limited via a route-scoped Throttler override to 5 requests / 15 min / IP. The global default elsewhere is 60 / min / IP. Login success/failure events are also written to audit_logs as AUTH_LOGIN_SUCCESS / AUTH_LOGIN_FAILED for SOC review.

4. Products Module -- Detailed Design

4.1 Class Diagram



4.2 Product Filter DTO

```
// backend/src/products/dto/find-products.dto.ts (conceptual)
export class FindProductsDto {
  @IsOptional() @IsString() q?: string;
  @IsOptional() @IsUUID() categoryId?: string;
  @IsOptional() @IsUUID() brandId?: string;
  @IsOptional() @IsUUID() designerId?: string;
  @IsOptional() @Type(() => Number) @Min(0) priceMin?: number;
  @IsOptional() @Type(() => Number) @Min(0) priceMax?: number;
  @IsOptional() @IsBoolean() inStock?: boolean;
  @IsOptional() @IsBoolean() onSale?: boolean;
  @IsOptional() @IsBoolean() isNew?: boolean;
  @IsOptional() @IsBoolean() isFeatured?: boolean;
  @IsOptional() @IsBoolean() isBestSeller?: boolean;
  @IsOptional() @IsIn(['priceAsc', 'priceDesc', 'newest', 'popular']) sort?: string;
  @IsOptional() @Type(() => Number) @Min(1) page?: number = 1;
  @IsOptional() @Type(() => Number) @Min(1) @Max(100) pageSize?: number = 24;
}
```

Soft-delete: every list query adds where: { deletedAt: null }.

Variant resolution: findOne joins productGroup -> products for sibling

variants and synthesises the variant axes (color/size/material) the UI

needs. A 60-second in-memory cache fronts category/brand/designer indices.

4.3 Product Response Structure

```
{
  "id": "uuid",
  "slug": "smart-luggage-28in-blue",
  "name": "Smart Luggage 28\"",
  "descriptionShort": "...",
  "descriptionLong": "...",
  "brand": { "id": "...", "name": "Solo", "slug": "solo" },
  "category": { "id": "...", "name": "Luggage", "slug": "luggage" },
  "pricing": {
    "priceExclVat": 950.00,
    "vatAmount": 47.50,
    "priceInclVat": 997.50,
    "salePriceInclVat": null,
    "currency": "AED"
  },
  "stock": { "qty": 12, "reservedQty": 0, "lowStockAlert": 3 },
  "flags": { "isNew": true, "isFeatured": false, "isBestSeller": true, "onSale": false },
  "images": [ { "url": "...", "alt": "..." } ],
  "group": {
    "id": "...",
    "axes": ["color", "size"],
    "siblings": [
      { "id": "...", "color": "blue", "size": "28in", "current": true },
      { "id": "...", "color": "black", "size": "28in", "current": false }
    ]
  }
}
```

5. Cart Module -- Detailed Design

Stateful cart per user (or guest cookie). All totals computed ****server-side** on every read****** -- never trust client-supplied subtotals.

Endpoint	Behaviour
GET /api/cart	Returns active cart + computed totals
POST /api/cart/items	Body { type=PRODUCT, itemId, quantity }; merge if exists
PATCH /api/cart/items/:id	Update quantity (0 = remove)
DELETE /api/cart/items/:id	Remove line
DELETE /api/cart	Clear
POST /api/cart/promo	Apply promo (validates active/expiry/usage)
DELETE /api/cart/promo	Remove promo
POST /api/cart/loyalty	Set loyaltyRedeemAed (capped at wallet balance)
DELETE /api/cart/loyalty	Clear loyalty redemption

Computation steps (in order): line subtotal -> cart subtotal (excl VAT) -> shipping (per site_settings.shipping) -> discount (promo) -> loyalty redemption -> VAT (subtotal x vatRate) -> grand total. The same engine is re-run inside OrdersService.create() so the snapshot fields written to the orders row match what the user saw.

6. Orders Module -- Detailed Design

6.1 Order Creation Flow

```

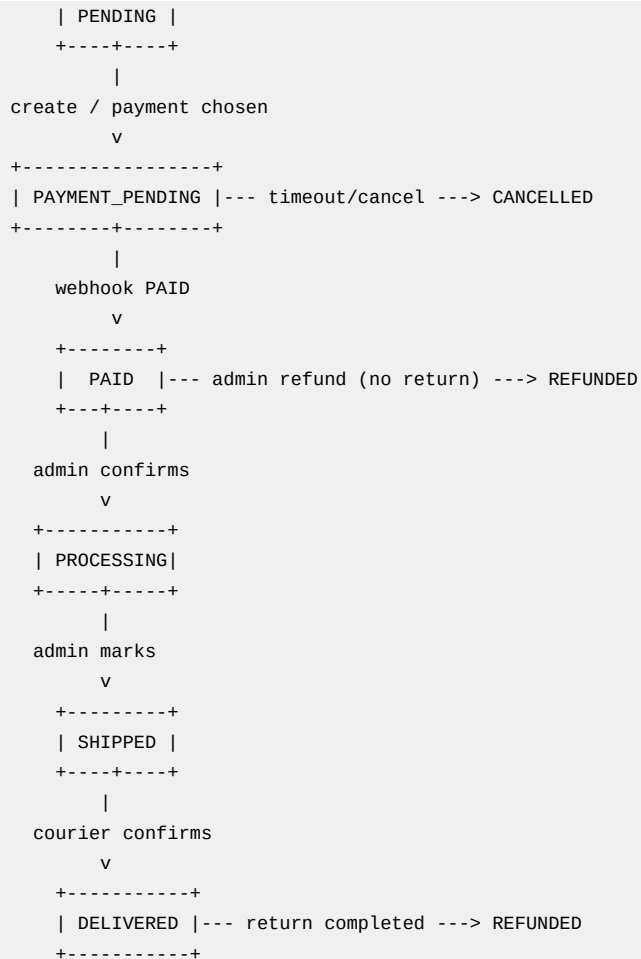
POST /api/orders {addressId, shippingMethod, paymentMethod, items, promoCode?, loyaltyAed?}
|
v
OrdersService.create()
|---> begin transaction
|   lock product rows (SELECT ... FOR UPDATE on each)
|   compute totals (re-run cart engine)
|   INSERT orders (status = PAYMENT_PENDING, snapshot all)
|   INSERT order_items (one per line, with VAT split)
|   UPDATE products SET reservedQty = reservedQty + qty
|   INSERT order_status_history
|   INSERT loyalty_transaction (PENDING earn)
|   INSERT audit_logs (ORDER_CREATED)
|---> commit
|
v
Return { orderId, total, paymentMethod }
|
v
Frontend redirects to provider (Stripe / Tabby / Tamara) or COD confirm

```

Snapshot fields written at creation: subtotalExclVat, vatAmount, vatRateSnapshot, shippingExclVat, shippingVat, discountExclVat, loyaltyRedeemAed, loyaltyEarnAed, totalInclVat, currencyCode.

6.2 Order Status State Machine

+-----+



Each transition writes an `order_status_history` row and a corresponding `audit_logs` row (`ORDER_PAID`, `ORDER_SHIPPED`, ...) via the global `AuditInterceptor`.

6.3 Invoice Generation

`OrdersService.generateInvoicePdf(orderId)` uses `pdfkit`:

1. Load order + items + customer + addresses.
 2. Stream a PDF: header (logo, invoice #, date), customer block, line items (name, qty, unit, VAT, total), totals block (subtotal, VAT, shipping, discount, loyalty, grand total), footer (TRN, terms).
 3. Stream returned to controller as `application/pdf`.
 4. Invoice number: `INV-{YYYY}-{monotonic}` from `invoice_sequence` table.
- Invoices are immutable post-issuance -- only their status may update.

7. Admin Module -- Detailed Design

7.1 Dashboard Statistics

`AdminDashboardController.GET /api/admin/dashboard` returns:

```
{
  "kpis": {
    "revenueToday": 12345.67,
```

```

"revenue7d": 87654.32,
"ordersToday": 42,
"newCustomers7d": 18,
"lowStockCount": 5
},
"revenueSeries": [{ "date": "2026-05-01", "revenue": 1200 }, ...],
"topProducts": [{ "id": "...", "name": "...", "soldQty": 42 }, ...],
"recentOrders": [{ "id": "...", "total": 250, "status": "PAID", "...": "..." }]
}

```

The frontend AdminDashboardPage renders these via Recharts

(<LineChart>, <BarChart>).

7.2 Reports Service

/api/admin/reports/*:

- sales-summary -- totals by period (day/week/month)
- revenue-period -- bar series for charting
- top-products -- paged ranking with revenue + units
- customers-ltv -- joined order totals per customer
- orders-by-status -- donut data
- low-stock -- products under lowStockAlert

All endpoints accept from, to, optional categoryId / brandId.

7.3 Audit Trail Module (NEW)

The audit-trail subsystem auto-logs every admin write action.

```

+-----+ +-----+
| AuditInterceptor | | AuditService |
+-----+ +-----+
| intercept(ctx, next) |---->| log({
|   - extract method, | |   userId, userEmail, |
|   path, params, body | |   action, entityType, |
|   - resolve entity id | |   entityId, |
|   - on success:      | |   ipAddress, userAgent, |
|   AuditService.log | |   metadata |
+-----+ | }) |
| |
| v |
+-----+ +-----+
| Reflector | | prisma.auditLog |
+-----+ | .create() |
| ^ |
| |
@AuditAction('PRODUCT_UPDATED')
on controller methods

```

Action name normalisation (regex first, then .toUpperCase()):

'productGroupUpdated' -> 'product_Group_Updated' -> 'PRODUCT_GROUP_UPDATED'.

Auto-logged controllers: every POST/PATCH/PUT/DELETE on

/api/admin/*. Auth events (`AUTH_LOGIN_SUCCESS/FAILED/REGISTERED/

LOGGED_OUT) are logged explicitly from AuthService`.

AuditController.GET /api/admin/audit accepts:

```

// AuditLogQueryDto (with class-validator)
@IsOptional() @IsString() page?: string;
@IsOptional() @IsString() limit?: string;
@IsOptional() @IsString() action?: string;

```



```
@IsOptional() @IsString() entityType?: string;
@IsOptional() @IsString() userId?: string;
@IsOptional() @IsString() userEmail?: string;
@IsOptional() @IsString() from?: string;
@IsOptional() @IsString() to?: string;
```

(All @IsString() because they arrive as URL query strings; conversion done in service.) ValidationPipe global forbidNonWhitelisted: true rejects unknown params, so all valid filters MUST be declared on the DTO.

8. Content/CMS Module -- Detailed Design

Table	Role
home_page_config	Singleton row for current home page
home_page_sections	Ordered sections (type + config JSONB)
category_landing_pages	Per-category landing config
landing_pages	Standalone landing pages (/landing/:slug)
banners	Schedulable banners (desktop + mobile media)
navigation_menu_items	Self-referencing tree (parentId)
product_collections	Curated collections (referenced by section type COLLECTION)
announcements	Top-bar messages, optional promo link
media_assets	Uploaded images metadata

ContentController (/api/content/*) returns assembled payloads

optimised for the storefront (single round-trip to render the home page).

Admin writes happen on `/api/admin/{banners,landing-pages,home-page,category-landing,navigation,collections,announcements}`.

Section types: HERO_BANNER, CATEGORY_TILES, BRAND_STRIP, COLLECTION, EDITORIAL, BEST_SELLERS, NEW_ARRIVALS, PROMO_STRIP, TESTIMONIALS.

9. Stripe Payment Module -- Detailed Design

```
SPA --(POST /api/stripe/create-payment-intent { orderId })--> StripeController
    |
    v
    StripeService
    |
    stripe.paymentIntents.create({
      amount: order.totalInclVat * 100,
      currency: 'aed',
      metadata: { orderId, userId },
      automatic_payment_methods: { enabled: true }
    })
    |
    v
    { clientSecret, paymentIntentId }

SPA --(stripe.confirmCardPayment(clientSecret))--> Stripe
Stripe --(POST /api/stripe/webhook, signed)--> StripeController.handleWebhook
    |
    v
    verify signature with STRIPE_WEBHOOK_SECRET
    |
```

```

      v
      UPSERT stripe_events (id PRIMARY KEY)
      if INSERT (new event):
        begin transaction:
          UPDATE orders SET status = PAID
          UPDATE products SET stockQty -= qty, reservedQty -= qty
          INSERT stock_movements (type=OUT)
          INSERT order_status_history
          UPDATE loyalty_transaction SET status=EARNED
          INSERT invoice (PDF generated lazily)
          INSERT audit_logs (ORDER_PAID)
        commit

```

Idempotency: stripe_events.id PK = Stripe event id. Replay = no-op.

10. Tabby & Tamara BNPL Modules

Both follow the same pattern as Stripe but with provider-specific session APIs and HMAC-signed webhooks.

Endpoint	Tabby	Tamara
Create session	POST /api/tabby/checkout	POST /api/tamara/checkout
Provider redirect	tabby.com/checkout/...	tamara.co/checkout/...
Webhook	POST /api/tabby/webhook	POST /api/tamara/webhook
Verification	TABBY_WEBHOOK_SECRET HMAC	TAMARA_NOTIFICATION_TOKEN HMAC
Idempotency	tabby_events table	tamara_events table

PAID transition writes the same audit + stock + loyalty + invoice fan-out.

11. Media Upload Module -- Detailed Design (Azure Blob Storage)

```

POST /api/media/upload  (multipart, ADMIN+)
|
v
MediaController.upload() -- uses @nestjs/platform-express FileInterceptor
|
v
MediaService.upload(file)
|---> sharp(file.buffer)
|      .resize(THUMB_W).jpeg() -> thumb buffer
|      .resize(MEDIUM_W).jpeg() -> medium buffer
|      .resize(LARGE_W).jpeg() -> large buffer
|---> blobServiceClient
|      .getContainerClient('media')
|      .getBlockBlobClient(key + '/original' | '/thumb' | ...)
|      .uploadData(buffer, { blobHTTPHeaders: { blobContentType } })
|---> prisma.mediaAsset.create({ data: { ...urls, mime, size, alt } })
|
v
return { id, urls: { original, thumb, medium, large } }

```

Auth: BlobAccountKey from env, or User-Assigned Managed Identity via DefaultAzureCredential in production.

12. Favorites Module -- Detailed Design

Endpoint	Behaviour
GET /api/favorites	List with paginated product summaries
POST /api/favorites { productId }	Idempotent insert (ON CONFLICT DO NOTHING)
DELETE /api/favorites/:productId	Remove

Product must be active and not soft-deleted; otherwise 404.

13. Promo Code Module -- Detailed Design

promo_codes columns: code (unique), type (PERCENT/FIXED), value, minOrderAmount, maxUses, maxUsesPerUser, usedCount, startsAt, endsAt, isActive.

Validation engine (PromosService.validate(code, cart, userId)):

1. Code exists and isActive.
2. now between startsAt..endsAt.
3. cart.subtotalExclVat >= minOrderAmount.
4. usedCount < maxUses.
5. userPromoUsages(userId, promoId) < maxUsesPerUser.

Redemption (inside order create transaction): atomic

UPDATE promo_codes SET usedCount = usedCount + 1 WHERE id=? AND usedCount < maxUses

returning rows; 0 -> throw ConflictException.

14. Loyalty Module -- Detailed Design

Table	Role
loyalty_wallets	One row per user (balanceAed, lifetimeEarnedAed)
loyalty_transactions	Ledger (EARNED, REDEEMED, ADJUSTED, EXPIRED, PENDING)

Earn rate (configurable in site_settings.loyalty.earnRate): default

2% of subtotalExclVat, posted as PENDING at order create and

flipped to EARNED at PAID. Refund via LOYALTY_AED posts a new

EARNED row equal to the refunded amount.

Redemption cap: min(walletBalance, cartSubtotal x maxRedeemPct) where maxRedeemPct defaults to 0.50.

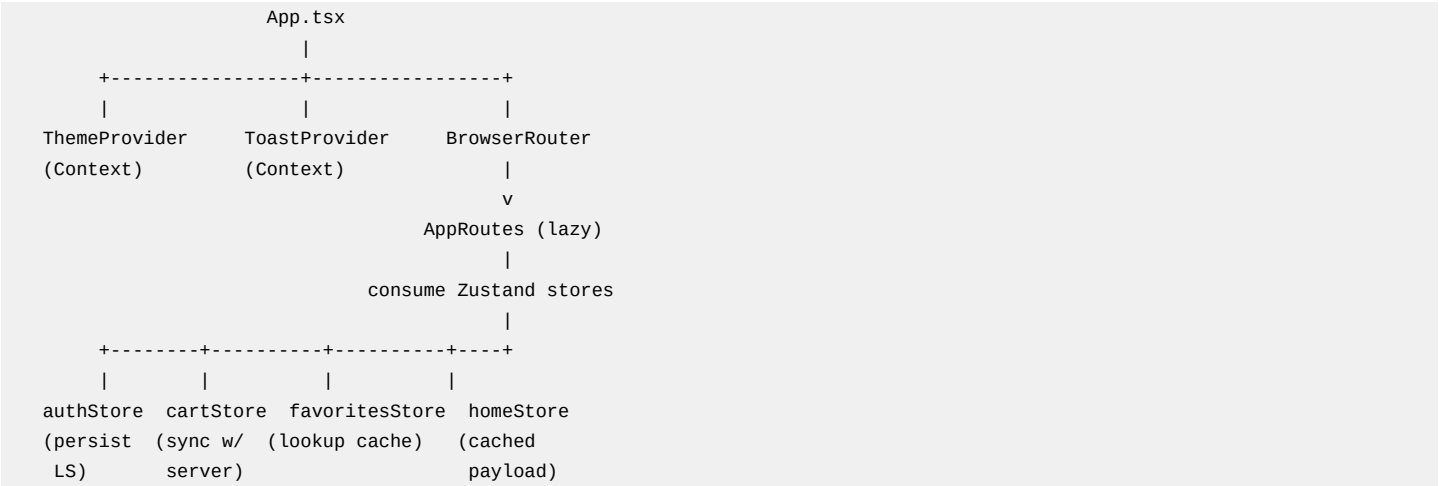
Expiry: monthly cron job marks transactions older than the configured TTL as EXPIRED and decrements balanceAed accordingly.

15. Frontend State Management Design (React)

15.1 Provider/Store Architecture

State is split between Zustand stores (cross-page persistent state)

and React Context (UI-only ephemeral state).



Store	Source File	Persisted?	Responsibility
authStore	src/stores/authStore.ts	yes (localStorage)	user, tokens, login(), logout(), refresh()
cartStore	src/stores/cartStore.ts	partial (id only)	Active cart, totals, optimistic add/update/remove with server reconciliation
favoritesStore	src/stores/favoritesStore.ts	yes (id set)	Favorite product IDs and detail cache
homeStore	src/stores/homeStore.ts	session memory	Cached /api/content/home payload + 5-min TTL
ThemeContext	src/contexts/ThemeContext.tsx	no	Light/dark -- UI only
ToastContext	src/contexts/ToastContext.tsx	no	Toast notifications

Stores never call fetch directly -- they delegate to API modules in `src/api/`.

15.2 API Client Design

`src/lib/apiClient.ts` (conceptual):

```
const baseUrl = import.meta.env.VITE_API_URL; // ".../api"

export async function apiFetch<T>({
  path: string,
  init: RequestInit = {},
}): Promise<T> {
  const accessToken = useAuthStore.getState().tokens?.accessToken;
  const headers: HeadersInit = {
    'Content-Type': 'application/json',
    ...(init.headers ?? {}),
    ...(accessToken ? { Authorization: `Bearer ${accessToken}` } : {}),
  };

  let res = await fetch(`${baseUrl}${path}`, { ...init, headers });

  // Single-shot refresh on 401
  if (res.status === 401 && accessToken) {
    const ok = await useAuthStore.getState().refresh();
    if (!ok) {
      useAuthStore.getState().logout();
      window.location.assign('/auth/login');
      throw new ApiError(401, 'Session expired');
    }
    const newToken = useAuthStore.getState().tokens!.accessToken;
    res = await fetch(`${baseUrl}${path}`, {
      ...init,
      headers: { ...headers, Authorization: `Bearer ${newToken}` },
    });
  }
}
```

```
    });
  }

  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new ApiError(res.status, body.message ?? res.statusText, body);
  }
  return res.json() as Promise<T>;
}
```

Per-domain modules (api/auth.ts, api/products.ts, api/orders.ts, api/admin.ts, ...) export typed wrappers. Example:

```
// src/api/admin.ts
export const adminApi = {
  getAuditLogs: (q: AuditLogQuery) =>
    apiFetch<PagedResult<AuditLog>>(`
    /admin/audit?${new URLSearchParams(q as any).toString()}`,
    ),
  // ...
};
```

16. Database Schema -- Key Tables

16.1 Core Tables with Column Detail

users

email	text	UNIQUE NOT NULL	
passwordHash	text	NOT NULL	-- argon2id encoded string
firstName	text		
lastName	text		
phone	text		
role	enum	NOT NULL DEFAULT 'CUSTOMER'	-- CUSTOMER ADMIN SUPER_ADMIN
emailVerified	boolean	NOT NULL DEFAULT false	
createdAt	timestampz	NOT NULL DEFAULT now()	
updatedAt	timestampz	NOT NULL	
deletedAt	timestampz		-- soft delete

products

slug	text	UNIQUE NOT NULL	
sku	text	UNIQUE NOT NULL	
productGroupId	uuid	FK -> product_groups.id	
brandId	uuid	FK -> brands.id	
categoryId	uuid	FK -> categories.id	
designerId	uuid	FK -> designers.id	
name	text	NOT NULL	
descriptionShort	text		
descriptionLong	text		
stockQty	int	NOT NULL DEFAULT 0	
reservedQty	int	NOT NULL DEFAULT 0	
lowStockAlert	int	NOT NULL DEFAULT 5	
isActive	boolean	NOT NULL DEFAULT true	
isNew	boolean	NOT NULL DEFAULT false	
isFeatured	boolean	NOT NULL DEFAULT false	
isBestSeller	boolean	NOT NULL DEFAULT false	
createdAt	timestampz	NOT NULL DEFAULT now()	
updatedAt	timestampz	NOT NULL	
deletedAt	timestampz		-- soft delete tombstone

orders

```

userId      uuid      FK -> users.id
status      enum      NOT NULL    -- see state machine
paymentMethod  enum      NOT NULL    -- STRIPE | TABBY | TAMARA | COD
subtotalExclVat  numeric(12,2) NOT NULL
vatAmount      numeric(12,2) NOT NULL
vatRateSnapshot  numeric(5,4)  NOT NULL
shippingExclVat  numeric(12,2) NOT NULL
shippingVat      numeric(12,2) NOT NULL
discountExclVat  numeric(12,2) NOT NULL DEFAULT 0
loyaltyRedeemAed  numeric(12,2) NOT NULL DEFAULT 0
loyaltyEarnAed    numeric(12,2) NOT NULL DEFAULT 0
totalInclVat     numeric(12,2) NOT NULL
currencyCode     text       NOT NULL DEFAULT 'AED'
addressBillingId  uuid
addressShippingId  uuid
createdAt        timestampz NOT NULL DEFAULT now()
updatedAt        timestampz NOT NULL

```

id

16.2 Indexes

Table	Index	Reason
products	(slug), (sku), (categoryId, isActive), (brandId, isActive), (deletedAt)	listing, lookup, soft-delete filter
orders	(userId, createdAt DESC), (status, createdAt DESC)	account orders, admin queue
order_items	(orderId), (productId)	join + reporting
refresh_tokens	(userId), (tokenHash)	login fan-out, rotation lookup
audit_logs	(createdAt DESC), (userId, createdAt DESC), (action), (entityType, entityId)	the audit page
stripe_events	(id) PK	idempotency

16.3 audit_logs Table

```

id          uuid      PK
userId      uuid      FK -> users.id
userEmail   text
action      text       NOT NULL    -- PRODUCT_CREATED, ORDER_PAID, ...
entityType  text          -- 'Product', 'Order', 'User', ...
entityId    text          -- usually the uuid
ipAddress   text
userAgent   text
metadata    jsonb              -- request body / before-after diff
createdAt   timestampz NOT NULL DEFAULT now()

```

Retention: 90 days hot in PG; older rows are exported nightly to cold storage and pruned. Append-only -- there is no UPDATE / DELETE API.

17. Error Handling Strategy

Global `HttpExceptionHandler` standardises all responses to:

```

{
  "statusCode": 422,
  "error": "Unprocessable Entity",
  "message": ["price must be a positive number"],
  "timestamp": "2026-05-12T10:11:12.345Z",
  "path": "/api/admin/products"
}

```

HTT	When
400	Bad request body / params
401	Missing or invalid JWT
403	Authenticated but role lacks permission
404	Entity not found (or soft-deleted)
409	State conflict (out-of-stock race, duplicate slug, promo race)
422	Validation error (class-validator violations)
429	Rate limit exceeded
500	Unhandled -- Pino + App Insights capture stack

Frontend: every API call surfaces an `ApiError` with status, message, details. Pages render banners (red for 4xx/5xx) -- e.g., the `AdminAuditLogPage` shows a banner instead of swallowing errors silently.

18. VAT Calculation Logic

UAE VAT rate 5% is a snapshot per order (`vatRateSnapshot`), not a runtime constant -- so historical orders remain correct if the rate changes (e.g., KSA expansion at 15%).

Per-line:

```
priceInclVat = priceExclVat * (1 + vatRate)
vatAmount   = priceExclVat *      vatRate
lineSubtotal = priceExclVat * qty
lineVat      = vatAmount      * qty
lineTotal    = priceInclVat * qty
```

Per-order:

```
subtotalExclVat = SUM(lineSubtotal)
vatAmount       = SUM(lineVat)
shippingExclVat = settings.shipping(method).priceExclVat
shippingVat     = shippingExclVat * vatRate
discountExclVat = promo.evaluate(subtotalExclVat)
loyaltyRedeem   = min(walletBalance, subtotalExclVat * maxRedeemPct)
totalInclVat    = (subtotalExclVat - discountExclVat - loyaltyRedeem)
                + vatAmount + shippingExclVat + shippingVat
```

Stored on the order at creation; never recomputed after PAID. Invoice PDF reads these snapshot fields verbatim, ensuring legal VAT compliance.

End of Low Level Design.